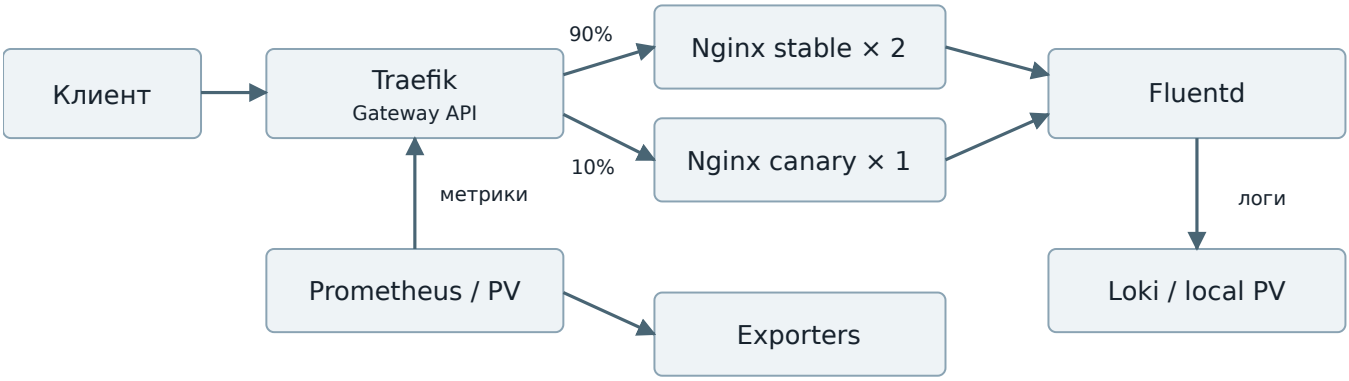
Self-Verifying Kubernetes Stack

Автоматическая установка и проверка веб-сервиса

Nginx возвращает `Hello World!` через Kubernetes Gateway API. Prometheus собирает метрики, Fluentd передает access/error-логи в Loki. Сквозная проверка связывает HTTP-ответ и запись лога по Request-ID и отдельно подтверждает прирост HTTP-счетчика.



На схеме показан canary-выпуск 90/10. После установки и отката stable получает 100% трафика. Поддерживаются dual-stack с основным IPv6 и отдельный режим IPv4 без IPv6.

Слой	Состав и версии
ОС и Kubernetes	Ubuntu 24.04 LTS amd64, проверено на 24.04.5; Kubernetes / kubeadm / kubelet / kubectl 1.34.3
Runtime и сеть	containerd 1.7.28; runc 1.3.3; CNI plugins 1.8.0; Calico 3.31.2, VXLAN
Gateway и приложение	Gateway API 1.4.0; Traefik 3.6.2, Helm chart 37.4.0; Nginx unprivileged 1.28.0-alpine
Метрики	Helm chart kube-prometheus-stack 80.6.0; Operator 0.87.1; Prometheus 3.8.1; node exporter 1.10.2; kube state metrics 2.17.0
Логи	Fluentd 1.19.0, образ grafana/fluent-plugin-loki:3.6.2; Loki 3.6.2
Автоматизация	Ansible core 2.19.3, Helm 3.19.0, Python, Bash и Make

> Проверенная конфигурация: одна VM, 4 vCPU, 8 GiB RAM, диск 60 GiB. Нужны SSH с sudo без пароля, IPv4 (для dual-stack также IPv6), DNS/NTP и доступ к публичным репозиториям пакетов и образов. Облачные сервисы и инфраструктура автора не требуются.

Состав и установка: README.md. Версии и контрольные суммы: config/versions.yaml и config/images.lock.yaml.

Обязательные компоненты

Из корня репозитория скопируйте подходящий пример из `config/environments/` в `config/environments/ubuntu.local.yaml`, укажите адреса VM, SSH, DNS/NTP и свободные подсети. После проверки SSH выполните `bash scripts/deploy.sh --env ubuntu`. Скрипт устанавливает инструменты, настраивает ОС и Kubernetes, затем компоненты.

Компонент	Реализация и обоснование	Проверка
Kubernetes и Ubuntu	kubeadm на Ubuntu 24.04; Ansible настраивает containerd, Calico и каталоги данных. Такой состав воспроизводится на выделенной VM без привязки к облаку.	`make health ENV=ubuntu`: готовность узла, DNS, HTTP через Pod и Service.
Веб-приложение	Nginx с точным ответом `Hello World!`, JSON access-логом и stderr. Простое приложение дает однозначный критерий исправности.	`curl -i --noproxy '*' http://<VM_IPv4>:30080/`; ожидаются HTTP 200 и заданное тело.
Gateway API	Traefik использует GatewayClass, Gateway и HTTPRoute к Service приложения. NodePort обеспечивает доступ без внешнего балансировщика.	`make verify-release ENV=ubuntu`: статусы ресурсов и внешние HTTP-запросы.
Prometheus	kube-prometheus-stack, Operator и ServiceMonitor собирают Traefik, node exporter, kube state metrics и Prometheus. HTTP-счетчик подтверждает обработку нагрузки.	`make verify ENV=ubuntu`: targets, свежие samples и прирост счетчика. Ручные PromQL-запросы — в README.
Fluentd и Loki	Fluentd DaemonSet читает CRI stdout/stderr и передает записи в Loki. Централизованный поиск позволяет найти конкретный запрос по Request-ID.	Тот же `make verify` ищет access-запись и отдельную ошибку отсутствующего файла в Loki.
Автоматизация	Один deploy объединяет Ansible и Helm. Версии, SHA-256 и digest закреплены; повторный запуск проверяет принадлежность кластера и сохраняет ресурсы.	`make verify-idempotency ENV=ubuntu`: два повторных развертывания, сравнение ресурсов, PVC и Secrets.
Документация	README, архитектура, эксплуатация, описание проверок и примеры окружений находятся в Git. Все необходимые действия доступны без инфраструктуры автора.	Установка по README; ручные проверки HTTP, Prometheus и Loki с ожидаемыми результатами.

> Основная проверка после установки: `make verify-release ENV=ubuntu`. Она объединяет проверку приложения, сети, метрик, логов и TLS-маршрутов. Ошибки дают ненулевой код возврата; JSON-отчеты сохраняются в `artifacts/`.

Полные команды и диагностика: README.md и docs/operations.md.

Дополнительные возможности

Возможность	Реализация и обоснование	Проверка
IPv6 и IPv4	Режим dual-stack использует IPv6 первым; режим ipv4 работает без IPv6 в ядре. Выбор позволяет использовать стенд в разных сетях, сохраняя одну процедуру установки.	`make health` и `make verify` с `ENV=ubuntu`: адреса, DNS, EndpointSlices, HTTP и логи выбранных семейств.
Маршруты Gateway API	Четыре HTTPRoute: общий маршрут, hostname `hello.test` и пути к stable/canary с URLRewrite. Отдельные пути проверяют каждый backend при любом общем весе.	`make verify-release ENV=ubuntu`: hostname/path, заголовки маршрута и 404 для неизвестного пути.
HTTPS	Локальная CA и сертификат создаются автоматически, хранятся в Secret и сохраняются при повторном deploy. HTTPS не требует публичного домена.	Тот же `verify-release` проверяет сертификат, отказ для неверных CA и hostname.
Canary и откат	Два backend и веса 90/10. Окно 60 с, минимум 100 запросов к canary, максимум 1% 5xx; неверное тело или недостаток данных возвращают 100/0. Это ограничивает влияние ошибочного кандидата.	`make verify-canary ENV=ubuntu`: здоровый кандидат и пять отрицательных сценариев.
Наблюдаемость	HTTP-счетчики и длительность запросов, CPU/RAM узла, состояние Kubernetes и поиск логов в Loki. Один endpoint собирается через одно семейство IP, без удвоения счетчиков.	`make verify ENV=ubuntu`; targets, PromQL и LogQL в README.
Ограничения Nginx	UID 101, файловая система только для чтения, без capabilities, повышения привилегий и API token. Это сокращает доступ процесса к контейнеру и кластеру.	`make verify ENV=ubuntu` проверяет настройки и работающий процесс.
Восстановление	Обработка прерванного Helm upgrade, local PV для истории, позиции и буфер Fluentd на узле. Механизмы сохраняют состояние при замене Pod и кратком отказе.	`make demo ENV=ubuntu`; отдельно с внешней машины: `make demo ENV=ubuntu DEMO_ARGS="--section reboot"`.

Сценарии отказов предназначены для тестовой VM и выполняются последовательно. Они временно меняют состояние стенда и проверяют восстановление. `make evidence ENV=ubuntu` собирает JSON-отчеты в локальный архив.

Конфигурации: deploy/, ansible/, config/. Сценарии: scripts/ и tests/.

Результаты и ревью

Проверка	Подтвержденный результат
Чистая установка	IPv4 без IPv6: 6 мин 33 с. Dual-stack: 5 мин 12 с и 5 мин 45 с. Подготовка VM и SSH не включена.
Локальные тесты	195 прошли на Ubuntu с IPv6; на VM без IPv6 — 193 прошли и 2 пропущены. Helm lint/render обоих режимов прошли.
HTTP/TLS, метрики и логи	Проверены оба режима; TLS/Gateway — 140 проверок для IPv4 и 154 для dual-stack.
Повтор и восстановление	Два повторных deploy, canary/откат, прерывание выпуска, восстановление истории и буфера, перезагрузка — успешно в полных прогонах.

Состав прогонов и ограничения: docs/verification.md. Машинные сводки: docs/evidence/. Для кода с поддержкой IPv4 сохранены SHA-256 101 файла; повторная проверка существующего dual-stack-кластера этим кодом не включает установку и отказы.

Ревью работы

Главная особенность

Проверяется наблюдаемое поведение: точный HTTP-ответ, изменение счетчика, запись в Loki и восстановление после отказа. JSON-отчеты и ненулевой код при ошибке делают результат каждой проверки доступным для разбора.

Самое сложное решение

Для подтверждения переключения canary сравнивались два подхода: статусы HTTPRoute и фактически обслуживаемые ответы. Выбрано ожидание заголовка `X-Traffic-Split`, заданного вместе с весами, поскольку Traefik может применять конфигурацию позже обновления статуса; затем измеряется трафик без повторных попыток.

Ограничения

Один узел и local PV не обеспечивают HA или защиту от потери диска; резервного копирования нет. При длительном отказе возможны дубли и потери логов; локальная CA не имеет автоматической ротации. NetworkPolicy, Grafana/Alertmanager и CI не реализованы, откат работает в рамках выпуска.

Предложения по развитию — требование задания

Перечисленные ниже возможности не реализованы и не входят в результаты проверки.

- NetworkPolicy и уведомления: нужны правила сетевых потоков, тесты разрешений и приемник Alertmanager.
- CI с временным Kubernetes: нужен runner с ресурсами для VM и доступом к репозиториям зависимостей.
- Несколько узлов, отказоустойчивое хранилище и backup/restore: нужны дополнительные VM и отдельное хранилище; для телеком-сценариев — стенд отказов каждой IP-семьи.

Технические решения: docs/decisions.md. Соответствие заданию: docs/submission.md.